

InterprocessCommunication

Q) What is inter process communication? Explain types of IPC.

Inter process Communication-Processes executing may be either co-operative or independent processes.

- **Independent Processes** – processes that cannot affect other processes or be affected by other processes executing in the system.
- **Cooperating Processes**–processes that can affect other processes or be affected by other processes executing in the system.

Co-operation among processes are allowed for following reasons –

- **Information Sharing**-There may be several processes which need to access the same file. So the information must be accessible at the same time to all users.
- **Computation speed up**- Often a solution to a problem can be solved faster if the problem can be broken down into sub-tasks, which are solved simultaneously (particularly when multiple Processors are involved.)
- **Modularity**-A system can be divided into cooperating modules and executed by sending information among one another.
- **Convenience**-Even a single user can work on multiple asks by information sharing.

Cooperating processes require some type of inter-process communication. This is allowed by two models:

1. Shared Memory systems
2. Message passing systems.

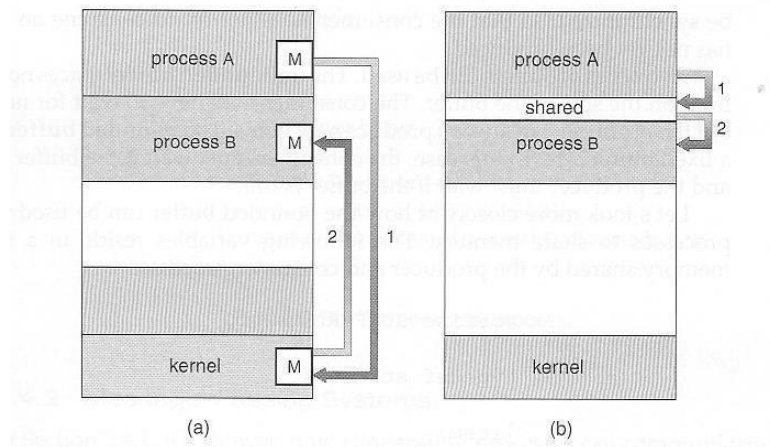


Figure 3.13 Communications models. (a) Message passing. (b) Shared memory.

S.No	Shared Memory	Message passing
1.	A region of memory is shared by communicating processes, into which the information is written and read	Message exchange is done among the processes by using objects.
2.	Useful for sending large block of data	Useful for sending small data.
3.	System call is used only to create Shared memory	System call is used during every Read and write operation.
4.	Message is sent faster, as there are no System calls	Message is communicated slowly.

- **Shared Memory** is faster once it is set up, because no system calls are required and access occurs at normal memory speeds. Shared memory is generally preferable when large amounts of information must be shared quickly on the same computer.
- **Message Passing** requires system calls for every message transfer, and is therefore slower, but it is simpler to set up and works well across multiple computers. Message passing is generally preferable when the amount and/or frequency of data transfers is small.

Shared-Memory Systems

- A region of shared-memory is created within the address space of a process, which needs to communicate. Other process that needs to communicate uses this shared memory.

- The form of data and position of creating shared memory area is decided by the process. Generally, a few messages must be passed back and forth between the cooperating processes first in order to set up and coordinate the shared memory access.
- The process should take care that the two processes will not write the data to the shared memory at the same time.

DEADLOCK CHARACTERIZATION

Necessary Conditions

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

1. **Mutual exclusion:** At least one resource must be held in a non-sharable mode, that is, only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
2. **Hold and wait:** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
3. **No preemption:** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
4. **Circular wait:** A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , ..., P_{n-1} is waiting for a resource held by P_n and P_n is waiting for a resource held by P_0 .

METHODS FOR HANDLING DEADLOCKS

The deadlock problem can be handled in one of three ways:

1. Use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlocked state.
2. Allow the system to enter a deadlocked state, detect it, and recover.
3. Ignore the problem altogether and pretend that deadlocks never occur in the system.

To ensure that deadlocks never occur, the system can use either **deadlock prevention** or **deadlock avoidance**.

Deadlock prevention provides a set of methods for ensuring that at least one of the necessary conditions cannot hold. These methods prevent deadlocks by constraining how requests for resources can be made.

Deadlock-avoidance requires that the operating system be given in advance additional information

concerning which resources a process will request and use during its lifetime. With this additional knowledge, it can decide for each request whether or not the process should wait. To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process

DEADLOCK PREVENTION

Deadlock can be prevented by ensuring that at least one of the four necessary conditions cannot hold.

Mutual Exclusion

- The mutual-exclusion condition must hold for non-sharable resources. Sharable resources, do not require mutually exclusive access and thus cannot be involved in a deadlock.
- Ex: Read-only files are example of a sharable resource. If several processes attempt to open a read-only file at the same time, they can be granted simultaneous access to the file. A process never needs to wait for a sharable resource.

Hold and Wait

To ensure that the hold-and-wait condition never occurs in the system, then guarantee that, whenever a process requests a resource, it does not hold any other resources.

- One protocol that can be used requires each process to request and be allocated all its resources before it begins execution.

No Preemption

The third necessary condition for deadlocks is that there be no preemption of resources that have already been allocated.

To ensure that this condition does not hold, the following protocols can be used:

- If a process is holding some resources and requests another resource that cannot be immediately allocated to it, then all resources the process is currently holding are preempted.
- The preempted resources are added to the list of resources for which the process is waiting. The process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.

Circular Wait

One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each process requests resources in an increasing order of enumeration.

DEADLOCK AVOIDANCE

- To avoid deadlocks an additional information is required about how resources are to be requested. With the knowledge of the complete sequence of requests and releases for each

process, the system can decide for each request whether or not the process should wait in order to avoid a possible future deadlock

- Each request requires that in making this decision the system consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process

MEMORY MANAGEMENT

- Program must be brought into memory and placed within a process for it to be run.
- To improve both the utilization of the CPU and the speed of its response to users, a general-purpose computer must keep several processes in memory.
- OS provides Many memory-management schemes to improve memory utilization.
- Input queue or job queue – collection of processes on the disk that are waiting to be brought into memory to run the program.

Contiguous Memory Allocation

- The main memory must accommodate both the operating system and the various user processes.
- Therefore we need to allocate main memory in the most efficient way possible.
- The memory is usually divided into two partitions: one for the resident operating system and one for the user processes.
- In contiguous memory allocation, each process is contained in a single section of memory that is contiguous to the section containing the next process.
- Simplest methods for allocating memory are to divide memory into several fixed-sized partitions.
- Each partition may contain exactly one process. Thus, the degree of multiprogramming depends on number of partitions.

Contiguous Memory Allocation	Non-Contiguous Memory Allocation
It is easier for the OS to control.	It is difficult for the OS to control.
Overhead is minimum as not much address translations are there while executing a process.	More Overheads are there as there are more address translations.

Both Internal fragmentation and external fragmentation occurs in Contiguous memory allocation method.	Only External fragmentation occurs in Non-Contiguous memory allocation method.
It includes single partition allocation and multi-partition allocation.	It includes paging and segmentation.
Wastage of memory is there.	No memory wastage is there.
In contiguous memory allocation, swapped-in processes are arranged in the originally allocated space.	In non-contiguous memory allocation, swapped-in processes can be arranged in any place in the memory.
It is of two types: 1. Fixed(or static) partitioning 2. Dynamic partitioning	It is of five types: 1. Paging 2. Multilevel Paging 3. Inverted Paging 4. Segmentation 5. Segmented Paging
It could be visualized and implemented using Arrays.	It could be implemented using Linked Lists.
Degree of multiprogramming is fixed as fixed partitions	Degree of multiprogramming is not fixed

Paging

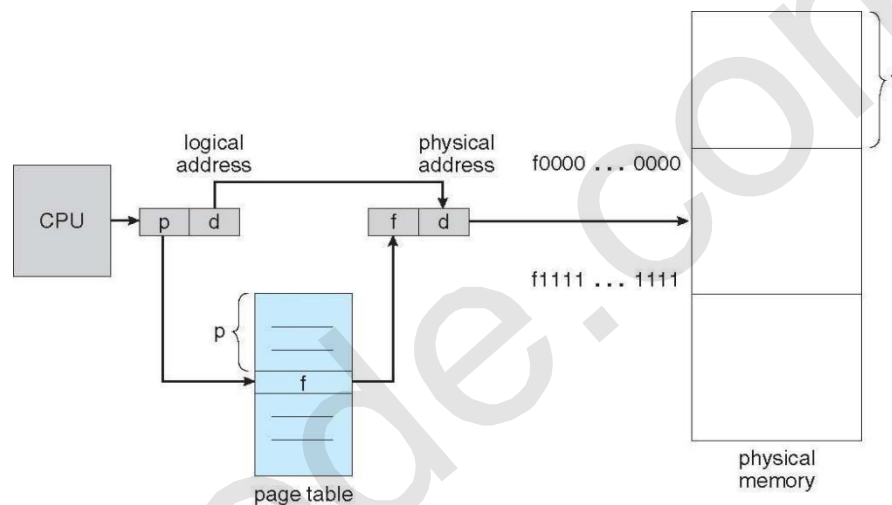
- Paging is a memory-management scheme.
- This permits the physical-address space of a process to be non-contiguous.
- This also solves the considerable problem of fitting memory-chunks of varying sizes onto the backing-store.
- Traditionally: Support for paging has been handled by hardware.
- Recent designs: The hardware & OS are closely integrated.

Basic Method of Paging

- The basic method for implementing paging involves breaking physical memory into fixed- sized blocks called frames and breaking logical memory into blocks of the same size called

pages.

- When a process is to be executed, its pages are loaded into any available memory frames from the backing store.



- Address generated by CPU is divided into 2 parts (Figure 2):
 - Page-number (p) is used as an index to the page-table. The page-table contains the base-address of each page in physical-memory.
 - Offset (d) is combined with the base-address to define the physical-address. This physical-address is sent to the memory-unit.
- The page table maps the page number to a frame number, to yield a physical address

Translation Look aside Buffer(TLB)

- A special, small, fast lookup hardware cache, called a translation look-aside buffer (TLB).
- Each entry in the TLB consists of two parts: a key (or tag) and a value.
- When the associative memory is presented with an item, the item is compared with all keys simultaneously. If the item is found, the corresponding value field is returned. The search is fast; the hardware, however, is expensive. Typically, the number of entries in a TLB is small, often numbering between 64 and 1,024.
- The TLB contains only a few of the page-table entries.

Working:

- When a logical-address is generated by the CPU, its page-number is presented to the TLB.
- If the page-number is found (TLB hit), its frame-number is immediately available and used to access memory
- If page-number is not in TLB (TLB miss), a memory-reference to page table must be made. The obtained frame-number can be used to access memory (Figure 1)

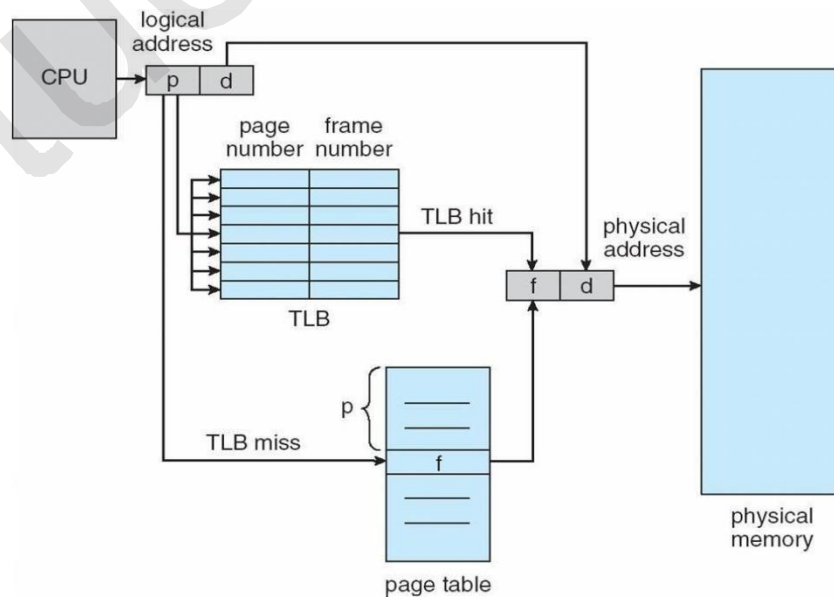


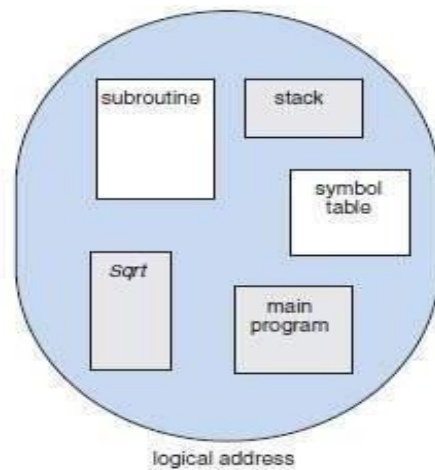
Figure 1: Paging hardware with TLB

- In addition, we add the page-number and frame-number to the TLB, so that they will be the next reference.
- If the TLB is already full of entries, the OS must select one for replacement.
- Percentage of times that a particular page-number is found in the TLB is called hit ratio.

Segmentation

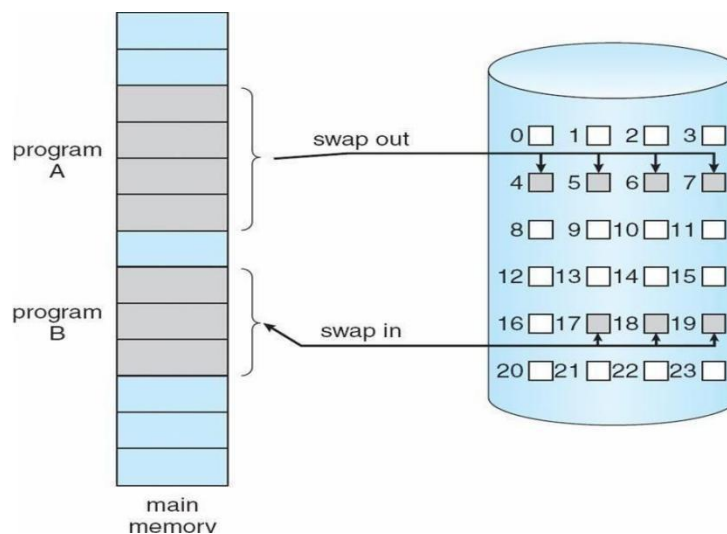
Basic Method of Segmentation

- This is a memory-management scheme that supports user-view of memory (Figure 1).
- A logical-address space is a collection of segments.
- Each segment has a name and a length.
- The addresses specify both segment-name and offset within the segment.
- Normally, the user-program is compiled, and the compiler automatically constructs segments reflecting the input program.
- For ex: The code, Global variables, The heap, from which memory is allocated, The stacks used by each thread, The standard C library



DEMAND PAGING

- A demand paging is similar to paging system with swapping when we want to execute a process we swap the process in to memory otherwise it will not be loaded in to memory.
- A swapper manipulates the entire processes, where as a pager manipulates individual pages of the process.
 - Bring a page into memory only when it is needed
 - Less I/O needed
 - Less memory needed
 - Faster response
 - More users
 - Page is needed $\Rightarrow$ reference to it
 - invalid reference $\Rightarrow$ abort
 - not-in-memory $\Rightarrow$ bring to memory
 - **Lazy swapper**– never swaps a page into memory unless page will be needed
 - Swapper that deals with pages is a **pager**.



Page Fault

If a page is needed that was not originally loaded up, then a **page fault trap** is generated.

Steps in Handling a Page Fault

1. The memory address requested is first checked, to make sure it was a valid memory request.
2. If the reference is to an invalid page, the process is terminated. Otherwise, if the page is not present in memory, it must be paged in.
3. A free frame is located, possibly from a free-frame list.
4. A disk operation is scheduled to bring in the necessary page from disk.
5. After the page is loaded to memory, the process's page table is updated with the new frame number, and the invalid bit is changed to indicate that this is now a valid page reference.
6. The instruction that caused the page fault must now be restarted from the beginning.

Multi-level queue Scheduling.

- Processes are divided into multiple queues based on type, priority, or characteristics.
- Each queue has its own scheduling algorithm, and queues are scheduled in a fixed priority order, where higher-priority queues are served first.

Synchronization?

Synchronization means coordinating multiple processes or threads so that they can work together correctly without interfering with each other. It ensures that shared resources are accessed in a controlled and orderly manner, preventing problems like data inconsistency or race conditions.

safe state in Deadlock?.

A safe state in deadlock avoidance is a condition of the system where all processes can finish their execution in some order without leading to a deadlock.

In a safe state, the system can always find a safe sequence that allows each process to obtain the required resources and complete successfully.

differences between Contiguous and Non Contiguous Allocation

Contiguous Memory Allocation	Non-Contiguous Memory Allocation
It is easier for the OS to control.	It is difficult for the OS to control.
Overhead is minimum as not much address translations are there while executing a process.	More Overheads are there as there are more address translations.

Thrashing.

Thrashing is a condition in which the operating system spends most of its time swapping pages in and out of memory rather than executing processes. It occurs when the demand for memory exceeds the available physical memory, causing extremely low CPU utilization and poor system performance.

synchronization essential in operating systems

- It prevents race conditions by ensuring that multiple processes or threads do not access shared data simultaneously in a conflicting manner.
- It maintains data consistency and system correctness when processes execute concurrently.